

Проект: Многослойный персептрон

Сагдуллин Амир 09 - 321

9 октября 2026 г.

Содержание

1	Введение	2
2	Первичный анализ данных (EDA)	2
2.1	Общая информация о датасете	2
2.2	Проверка качества данных	3
2.3	Баланс классов	4
2.4	Описательная статистика	4
2.5	Визуальный анализ признаков	4
2.6	Корреляционный анализ	6
3	Разделение данных	7
4	Архитектура нейронной сети	7
4.1	Прямой проход	7
4.2	Функция потерь	8
4.3	Обратный проход	8
4.4	Инициализация весов	8
5	Реализация	8
6	Обучение	9
6.1	Анализ	10
7	Прогнозирование	10
8	Выводы	11

1 Введение

Цель проекта - реализовать многослойный персептрон (MLP) с нуля на Python, без использования готовых библиотек машинного обучения (PyTorch, TensorFlow, Keras). В работе применяются только NumPy для линейной алгебры, pandas для загрузки данных и matplotlib для визуализации.

Задача - бинарная классификация объектов из датасета Breast Cancer Wisconsin Diagnostic: по 30 числовым признакам, описывающим ядро клетки, необходимо определить, является ли опухоль злокачественной (M) или доброкачественной (B).

2 Первичный анализ данных (EDA)

2.1 Общая информация о датасете

Датасет содержит 569 объектов и 32 столбца: идентификатор, метку класса (диагноз) и 30 числовых признаков. Признаки описывают три группы характеристик ядра клетки: средние значения (mean), стандартные ошибки (error) и худшие значения (worst) для десяти базовых параметров.

	id	diagnosis	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	...	worst radius	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	v
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	...	25.38	17.33	184.60	2019.0	0.1622	0.0	
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...	24.99	23.41	158.80	1956.0	0.1238	0.0	
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...	23.57	25.53	152.50	1709.0	0.1444	0.0	
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...	14.91	26.50	98.87	567.7	0.2098	0.0	
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...	22.54	16.67	152.20	1575.0	0.1374	0.0	

5 rows × 32 columns

Рис. 1: Первые строки датасета

2.2 Проверка качества данных

```
Пропусков в данных: 0
Полностью дублирующихся строк: 0
Дублирующихся id: 0

Типы данных:
float64      30
int64         1
object        1
Name: count, dtype: int64
```

Рис. 2: Проверка пропусков, дубликатов и типов данных

Пропусков нет, полностью дублирующихся строк нет, дублирующихся идентификаторов нет. Данные не требуют дополнительной очистки.

2.3 Баланс классов

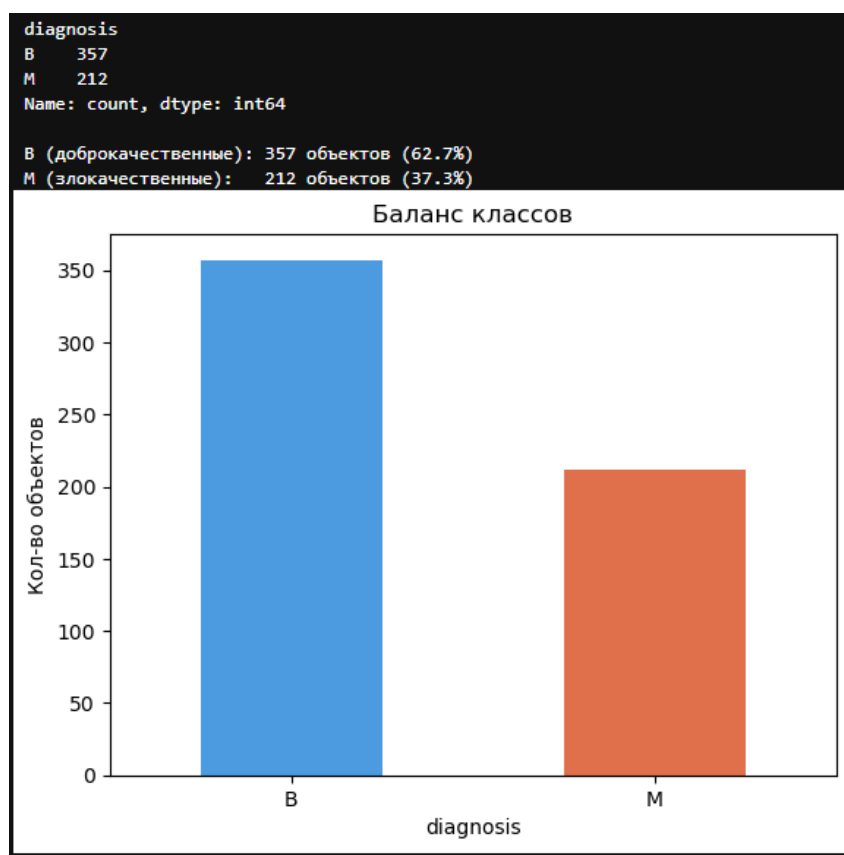


Рис. 3: Распределение классов

Класс В (доброкачественные) - 357 объектов (62.7%), класс М (злокачественные) - 212 объектов (37.3%). Наблюдается умеренный дисбаланс, поэтому при разделении данных используется стратификация.

2.4 Описательная статистика

Признаки имеют существенно разные масштабы: например, **mean area** варьируется в диапазоне 143.5-2501, а **mean smoothness** - в диапазоне 0.053-0.163. Это делает необходимой нормализацию перед обучением.

2.5 Визуальный анализ признаков

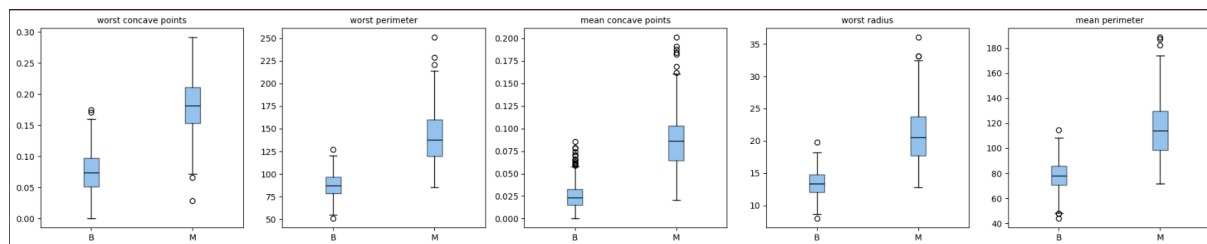


Рис. 4: Вохplot топ-5 признаков по корреляции с целевой переменной

Наиболее информативные признаки чётко разделяют классы: медиана класса М значительно выше, чем класса В. Встречаются выбросы, но они отражают реальные особенности данных (аномально крупные или мелкие ядра клеток), а не ошибки измерений, поэтому удалять их нецелесообразно.

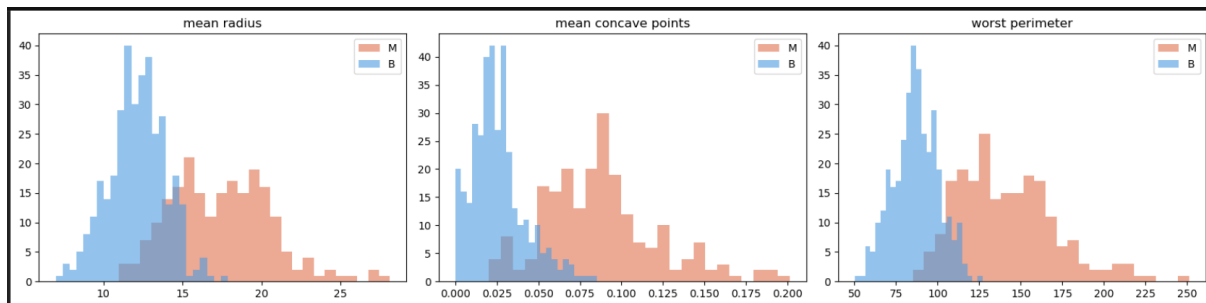


Рис. 5: Распределения значений трёх наиболее информативных признаков по классам

Гистограммы подтверждают результаты корреляционного анализа: для каждого из трёх признаков пики распределений классов В и М смещены относительно друг друга, что говорит о высокой разделяющей способности этих характеристик. При этом наблюдается перекрытие в средней части распределений - именно объекты из этой зоны и становятся сложными случаями для классификатора.

2.6 Корреляционный анализ

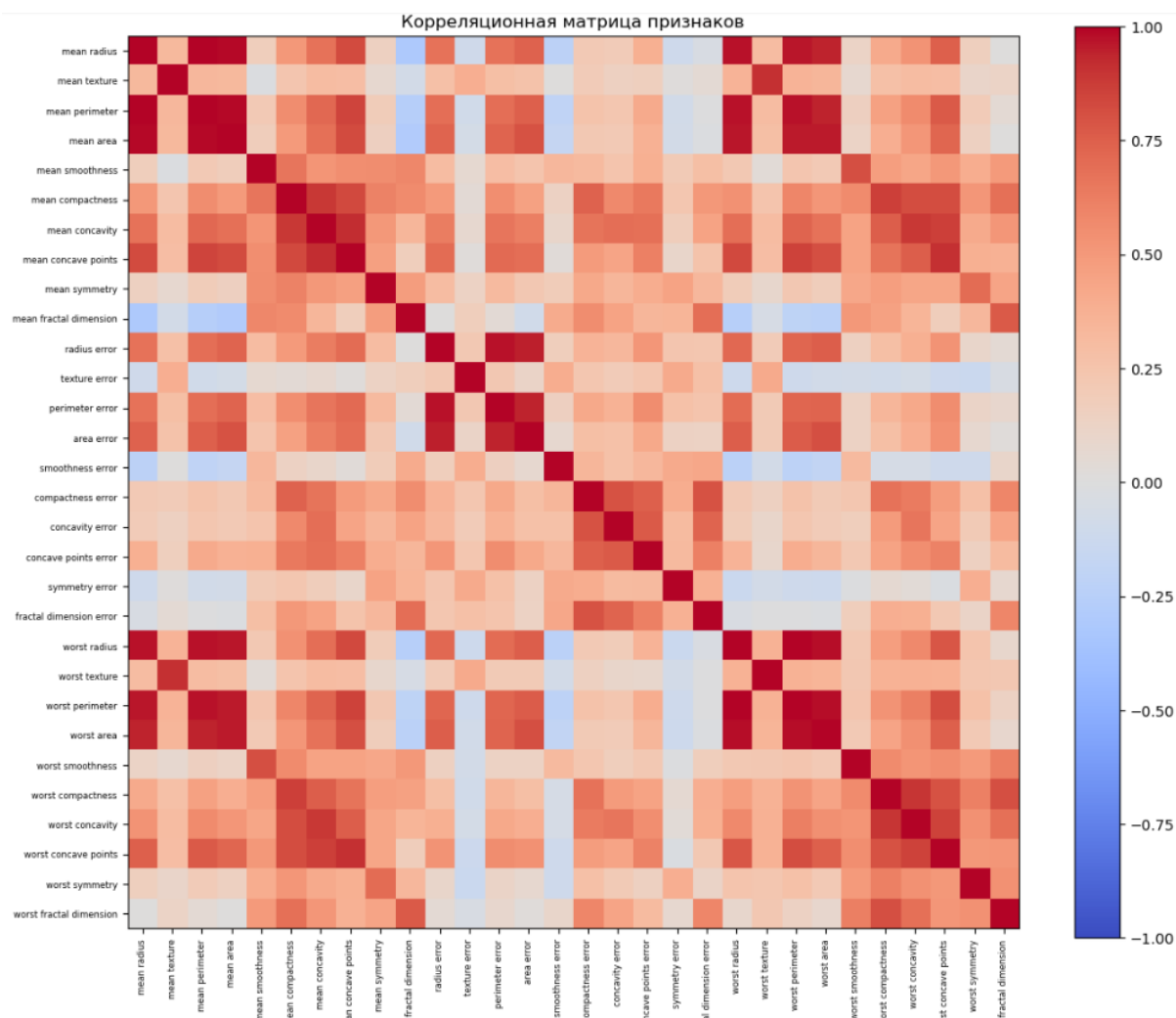


Рис. 6: Корреляционная матрица признаков

worst concave points	0.794
worst perimeter	0.783
mean concave points	0.777
worst radius	0.776
mean perimeter	0.743
worst area	0.734
mean radius	0.730
mean area	0.709
dtype:	float64

Рис. 7: Корреляция признаков с целевой переменной

Обнаружено 15 пар признаков с коэффициентом корреляции по модулю больше 0.95. Основные тройки: `radius`, `perimeter`, `area` - в каждой из групп (`mean/error/worst`) они почти линейно связаны между собой. Для MLP мультиколлинеарность не критична, поэтому удаление признаков не выполнялось.

Наибольшая корреляция с целевой переменной: `worst concave points` (0.794), `worst perimeter` (0.783), `mean concave points` (0.777).

3 Разделение данных

Данные разделены на обучающую и валидационную выборки в пропорции 80/20 со стратификацией по классу:

```
Train: shape=(456, 32)  B=286 (62.7%)  M=170 (37.3%)
Valid: shape=(113, 32)  B=71  (62.8%)  M=42  (37.2%)
```

Стратификация позволила сохранить пропорцию классов в обеих выборках. Случайное перемешивание выполнено с фиксированным `seed = 42` для воспроизводимости результатов.

4 Архитектура нейронной сети

4.1 Прямой проход

Для слоя с весами $W \in \mathbb{R}^{n_{in} \times n_{out}}$ и смещением $b \in \mathbb{R}^{1 \times n_{out}}$ прямой проход:

$$z = xW + b, \tag{1}$$

$$a = f(z), \tag{2}$$

где f — функция активации. В скрытых слоях используется сигмоида:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad (3)$$

на выходном слое — softmax для двух классов:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^2 e^{z_j}}. \quad (4)$$

4.2 Функция потерь

Используется бинарная кросс-энтропия (для one-hot представления двух классов):

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^2 y_{ik} \log \hat{p}_{ik}, \quad (5)$$

где y_{ik} — целевое значение, $\hat{p}_{ik}$ — предсказанная вероятность.

4.3 Обратный проход

Ключевое преимущество комбинации softmax + кросс-энтропия — упрощение градиента на выходном слое:

$$\frac{\partial L}{\partial z^{(\text{out})}} = \frac{1}{N} (\hat{p} - y). \quad (6)$$

Для скрытых слоёв с сигмной:

$$\frac{\partial L}{\partial z^{(l)}} = \frac{\partial L}{\partial a^{(l)}} \odot a^{(l)} \odot (1 - a^{(l)}), \quad (7)$$

$$\frac{\partial L}{\partial W^{(l)}} = (a^{(l-1)})^\top \frac{\partial L}{\partial z^{(l)}}, \quad \frac{\partial L}{\partial b^{(l)}} = \sum \frac{\partial L}{\partial z^{(l)}}. \quad (8)$$

4.4 Инициализация весов

Используется инициализация Хе:

$$W_{ij} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right). \quad (9)$$

5 Реализация

Проект состоит из четырёх модулей:

- `src/layers.py` - класс `DenseLayer` и функции активации (`sigmoid`, `relu`, `softmax`);
- `src/model.py` - сборка сети (`createNetwork`), обучение (`fit`), предсказание (`predict`), сохранение и загрузка модели;
- `src/split_data.py` - разделение данных;

- `src/train.py`, `src/predict.py` - командные интерфейсы для обучения и прогнозирования.

Архитектура сети задаётся через аргументы командной строки, что позволяет легко экспериментировать с числом и размером слоёв.

6 Обучение

Гиперпараметры:

- Архитектура: $30 \rightarrow 24 \rightarrow 24 \rightarrow 24 \rightarrow 2$
- Активации: `sigmoid`, `sigmoid`, `sigmoid`, `softmax`
- Функция потерь: бинарная кросс-энтропия
- Оптимизатор: `SGD`
- Learning rate: 0.0314
- Batch size: 8
- Количество эпох: 84
- Seed: 42

Команда запуска:

```
python src/train.py --layer 24 24 24 --epochs 84 --loss
    binaryCrossentropy \
        --batch_size 8 --learning_rate 0.0314
```

Лог обучения (фрагмент):

```
x_train shape : (456, 30)
x_valid shape : (113, 30)
epoch 01/84 - loss: 0.6558 - val_loss: 0.6572 - acc: 0.6272 - val_acc: 0.6283
epoch 10/84 - loss: 0.3080 - val_loss: 0.3244 - acc: 0.9364 - val_acc: 0.9381
epoch 20/84 - loss: 0.1054 - val_loss: 0.1209 - acc: 0.9781 - val_acc: 0.9735
epoch 30/84 - loss: 0.0710 - val_loss: 0.0925 - acc: 0.9868 - val_acc: 0.9735
epoch 40/84 - loss: 0.0588 - val_loss: 0.0872 - acc: 0.9868 - val_acc: 0.9735
epoch 50/84 - loss: 0.0525 - val_loss: 0.0859 - acc: 0.9890 - val_acc: 0.9735
epoch 60/84 - loss: 0.0483 - val_loss: 0.0867 - acc: 0.9912 - val_acc: 0.9646
epoch 70/84 - loss: 0.0453 - val_loss: 0.0909 - acc: 0.9912 - val_acc: 0.9646
epoch 80/84 - loss: 0.0429 - val_loss: 0.0911 - acc: 0.9912 - val_acc: 0.9558
epoch 84/84 - loss: 0.0421 - val_loss: 0.0943 - acc: 0.9912 - val_acc: 0.9558
> saving model 'saved_model.npy' to disk...
> learning curves saved to 'results/figures/learning_curves.png'
```

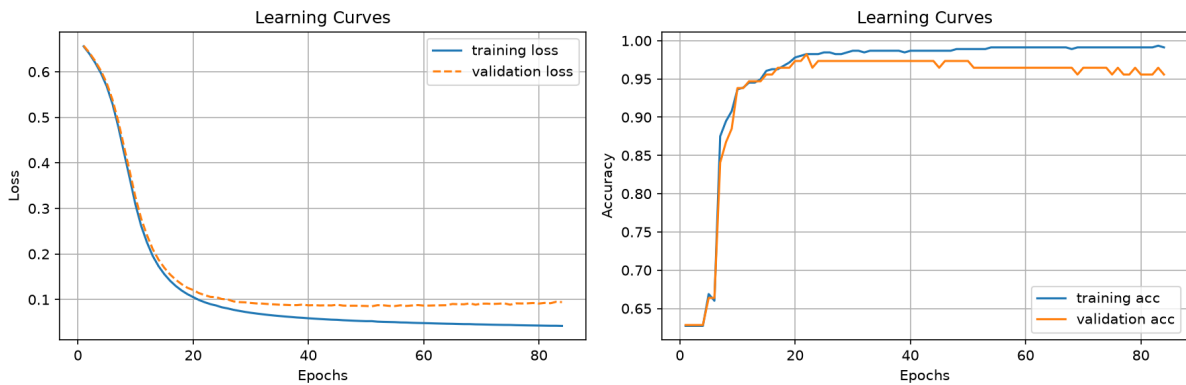


Рис. 8: Кривые обучения: loss (слева) и ассигасу (справа)

6.1 Анализ

Начальное значение ассигасу (0.627) совпадает с долей класса B в датасете — это baseline, который даёт модель, предсказывающая всегда класс большинства. После обучения ассигасу на валидации достигла 0.956, что существенно выше baseline.

Наблюдается умеренное переобучение: train loss (0.0421) меньше validation loss (0.0943) примерно в 2.2 раза, train accuracy (0.9912) превышает validation accuracy (0.9558) на 3.5 процентных пункта. Кривая валидации выходит на плато примерно к 30-й эпохе, дальнейшее обучение не даёт прироста.

7 Прогнозирование

Команда:

```
python src/predict.py --dataset data_split/data_validation.csv \
    --model_path saved_model.npy
```

Test samples: 113

Binary cross-entropy: 0.0943

Accuracy: 0.9558

First predictions:

```
#00: pred=M true=B [MISS] p(M)=0.755
#01: pred=B true=B [ok] p(M)=0.001
#02: pred=M true=M [ok] p(M)=0.994
#03: pred=M true=M [ok] p(M)=0.999
#04: pred=B true=B [ok] p(M)=0.002
#05: pred=M true=M [ok] p(M)=0.999
#06: pred=B true=B [ok] p(M)=0.078
#07: pred=M true=M [ok] p(M)=1.000
#08: pred=M true=B [MISS] p(M)=0.881
#09: pred=B true=B [ok] p(M)=0.011
```

Метрики на валидационной выборке полностью совпали с последней эпохой обучения (0.0943 и 0.9558), что подтверждает корректность сохранения и загрузки модели, а также правильное применение нормализации.

Обе ошибочных классификации относятся к типу false positive (модель предсказала M , тогда как истинный класс B). В контексте медицинской задачи это менее критично, чем false negative (пропуск злокачественной опухоли), однако всё равно требует внимания.

Большинство предсказаний уверенные: вероятность $p(M)$ либо меньше 0.1, либо больше 0.9. Это говорит о хорошем качестве разделяющей границы.

8 Выводы

В ходе работы реализован многослойный персептрон с нуля на NumPy. Основные результаты:

- Реализованы прямой и обратный проходы, функции активации, softmax, кросс-энтропия, инициализация He, SGD.
- Достигнута ассигасу 95.6% на валидационной выборке при 84 эпохах обучения.
- Модель корректно сохраняется и загружается, включая параметры нормализации.
- Наблюдается умеренное переобучение, которое можно уменьшить регуляризацией (dropout, weight decay) или уменьшением числа эпох/нейронов.
- Для дальнейшего улучшения можно применить оптимизатор Adam, BatchNorm или расширить датасет аугментациями.